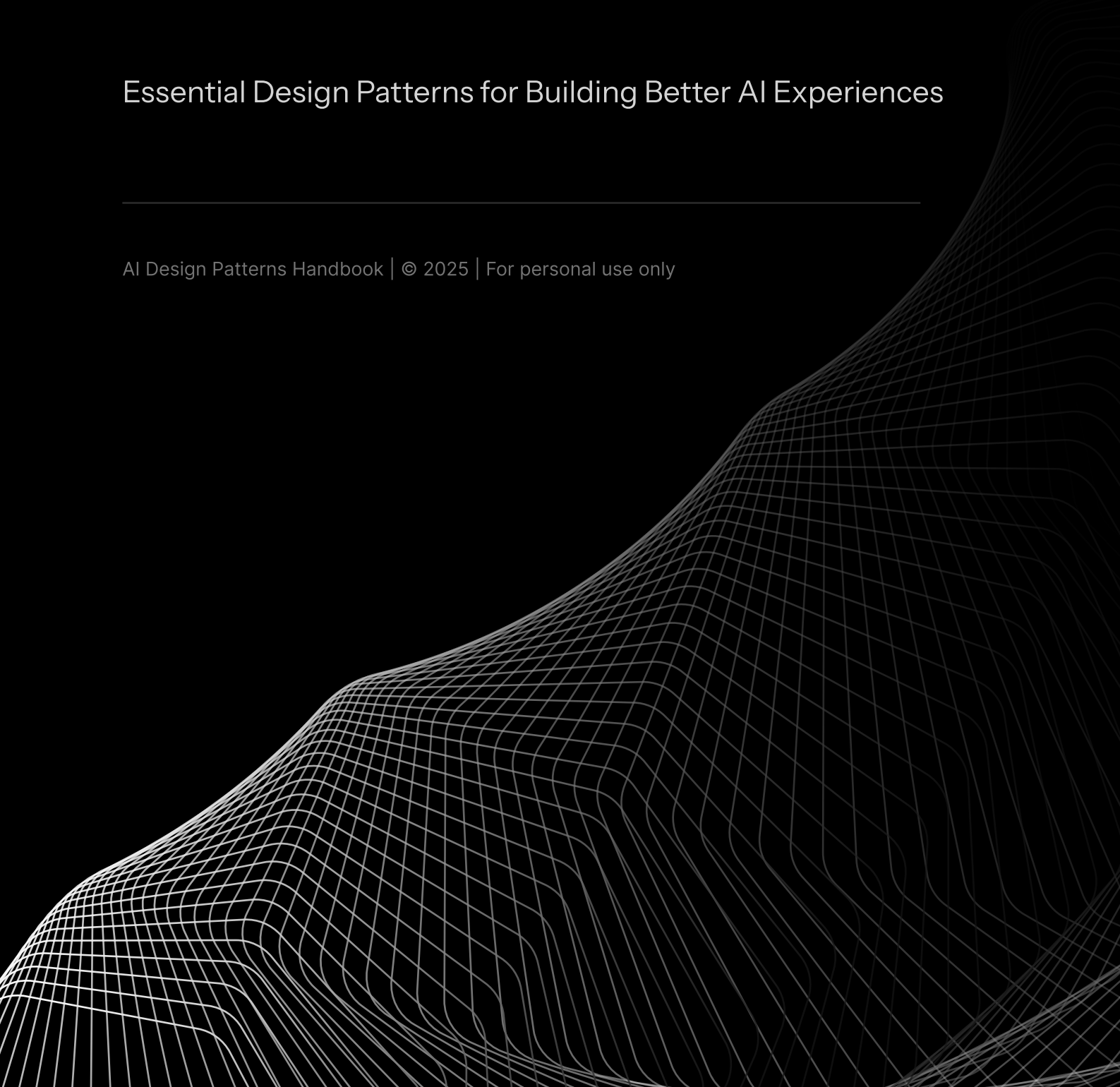


6 ESSENTIAL PATTERNS

AI Design Patterns

Essential Design Patterns for Building Better AI Experiences

AI Design Patterns Handbook | © 2025 | For personal use only



Contextual Assistance

Help users when they're actually stuck, not when you think they might be

The Idea

The best AI help is invisible until needed. Watch for signals: A pause, a mistake, a search. Offer suggestions at exactly that moment, then disappear if ignored. Like a helpful colleague who notices you're stuck, makes one suggestion, and steps back. Not someone interrupting every five minutes. This respects user autonomy and keeps them in flow. Detect the signal, make the suggestion, respect the choice.

When to Use

- ✓ User is paused, stuck, or searching
- ✓ Help is relevant to the current moment
- ✓ User can dismiss or ignore it

Products Doing It Right

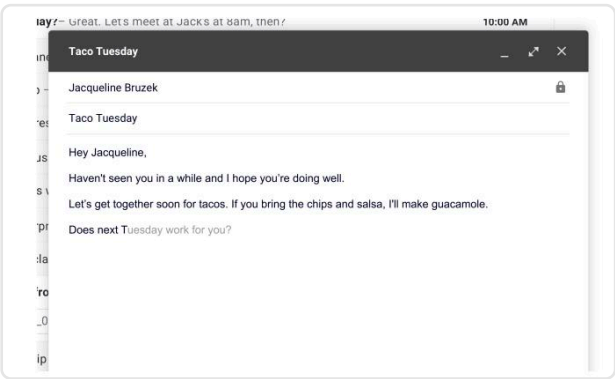
-  Gmail Smart Compose
Suggests text when you pause writing
-  GitHub Copilot
Suggests code when you pause typing
-  Notion AI
Offers help in relevant moments

Key Insight

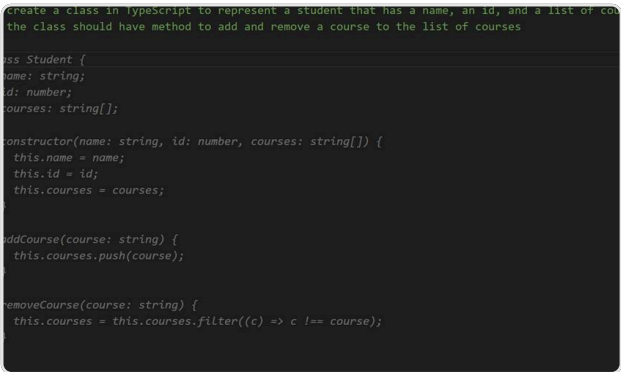
*Invisible until needed. Clear when presented.
Gone if rejected.*

DO ✓	DON'T ✕
Detect signals: pauses, mistakes, searches	"Ask AI" button everywhere
Offer as suggestion (not requirement)	Help even when user is performing well
Disappear if ignored	Force interaction

Examples



Gmail Smart Compose



GitHub Copilot

Confidence Visualization

PATTERN - 2

Show certainty levels so users know when to trust AI results

The Idea

AI can be wrong. Users forgive wrong answers if they see them coming. Show how sure you are. "94% confident," "42% sure," or "uncertain." This transforms trust from "Does the AI know?" to "Do I agree with its confidence level?" Users can then decide: Trust it, verify it, or skip it. Different domains need different thresholds. 87% for medical diagnosis? Risky. For detection tasks? Good. Let users see the number and decide

When to Use

- ✓ AI result has meaningful certainty variations
- ✓ High-stakes decisions or uncertain outcomes

DO ✓

Show confidence as %, labels, or visuals

Explain what confidence means at each level

Let users make informed decisions

Key Insight

Invisible until needed. Clear when presented.
Gone if rejected.

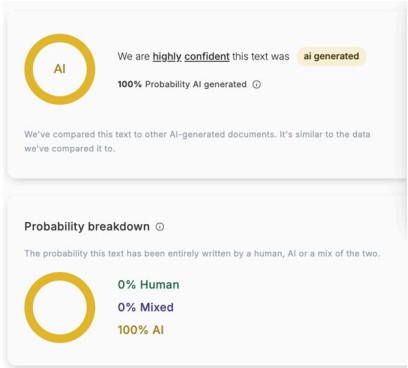
DON'T ✗

Hide uncertainty

Treat 75% and 99% confidence the same

Show confidence but no explanation

Examples



GPTZero shows confidence levels with visual gauges and percentages



AWS Rekognition displays confidence % for each detection

Error Recovery

Handle failures so users stay confident

PATTERN - 3

The Idea

AI will fail. The question is how you handle it. Be honest about what failed. Explain why and give recovery options. Users don't mind failures. They mind feeling helpless. Failures with clear recovery paths actually build trust. ChatGPT's "at capacity" message tells users the system is honest about limits. Silent failures make it seem broken. Show what failed, explain why, and let users choose the next step.

When to Use

- ✓ AI might fail or produce unexpected results
- ✓ Users need clear explanation when it fails
- ✓ Multiple recovery options are available

Products Doing It Right

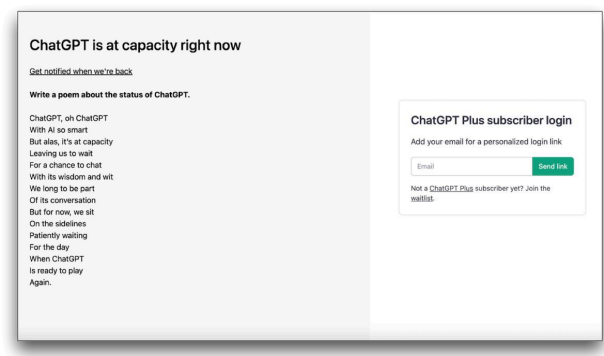
-  ChatGPT
"At capacity" with clear retry options
-  Grammarly
Shows explanations + dismissal options
-  Google
"Did you mean?" (ask, don't assume)

Key Insight

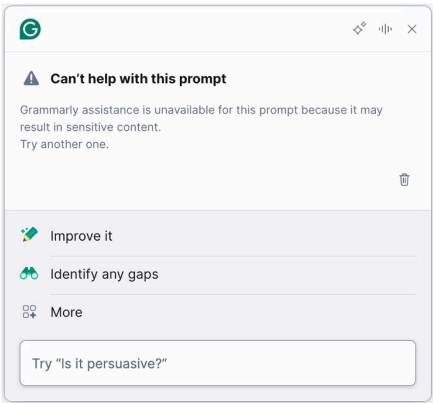
Users forgive wrong answers. They don't forgive feeling helpless.

DO ✓	DON'T ✕
Be honest about failure	Silent failures
Explain what went wrong	Generic errors ("Something went wrong")
Recovery options (retry, override, manual)	Auto-correct without telling

Examples



ChatGPT at capacity message



Grammarly provides explanations and fallback functionality

Privacy-First Design

PATTERN - 4

Make users feel safe sharing data with AI

The Idea

Users don't hate AI. They hate surprise. Be transparent by default. "We collect location data to improve search. It stays encrypted on-device. Deleted after 30 days. Here's how to turn it off." Not hidden three menus deep. Not vague language. Apple works because users know data stays on-device. DuckDuckGo works because users know they're not tracked. Transparency builds trust and gives you competitive advantage

Products Doing It Right

-  Apple
AI runs on-device, not on servers
-  DuckDuckGo
Anonymous AI chat (no logs, no profiles)
-  Signal
End-to-end encrypted AI features

When to Use

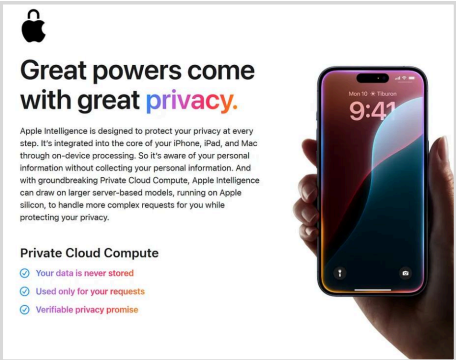
- ✓ AI collects or processes user data
- ✓ Users need to understand data usage
- ✓ Users can control or opt out

Key Insight

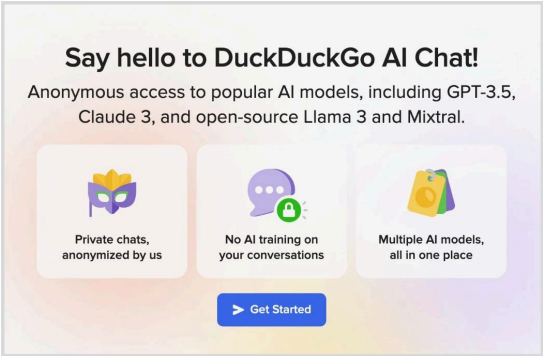
Privacy isn't sacrifice. It's competitive advantage. Users trust and use AI more when they feel in control.

DO ✓	DON'T ✗
Explain what data you collect and why	Vague language ("improve experience")
Get consent (ask first, don't assume)	Hidden settings
Show user controls for data	Impossible to opt out

Examples



Apple's on-device intelligence keeps data private



DuckDuckGo anonymous AI chat without tracking

Explainable AI

Show users why AI made its decision

The Idea

Black boxes are untrustworthy. Show users why you made a decision or recommendation in plain language. Claude shows step-by-step thinking. Perplexity cites sources. Users can then verify, evaluate, and understand the reasoning. Explainability isn't about showing all 47 factors. It's about highlighting the 2-3 most important ones and showing where information came from. Users trust AI more when they understand it, even if they sometimes disagree

When to Use

- ✓ AI makes recommendations or decisions
- ✓ Users need to evaluate the reasoning
- ✓ You can explain in simple language

DO ✓

Explain in plain language (not technical jargon)

Show top 2-3 reasons (not all 47 factors)

Be accurate (users double-check)

Key Insight

Explanations build confidence. Users trust AI that shows its thinking, even when they disagree

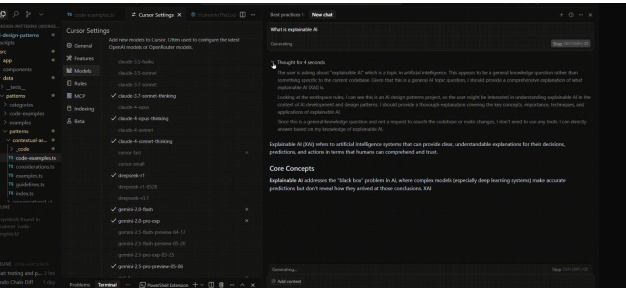
DON'T ✗

Black box output (no explanation)

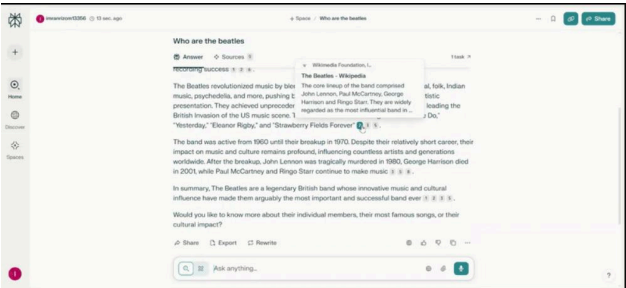
Wrong explanation (users notice)

Obvious statement ("Search results for 'apple'")

Examples



Claude shows step-by-step thinking and reasoning for conclusions



Perplexity shows exact sources for verification

Progressive Disclosure

Show simple first, power later, never overwhelm

The Idea

Beginners and power users have different needs. Don't show all options at once. Start with the core action. One clear button. Then reveal advanced features as users ask for them. Loom shows basic editing first, then AI features appear on demand. Google Docs reveals features as you use it. Make the simple path obvious. Let power users find their tools. Same feature. Different complexity curves for different expertise levels.

Products Doing It Right

-  Loom AI
Basic video editing first, advanced AI features on demand
-  Superhuman AI Email
Progressively reveals features like Smart Compose as you engage

When to Use

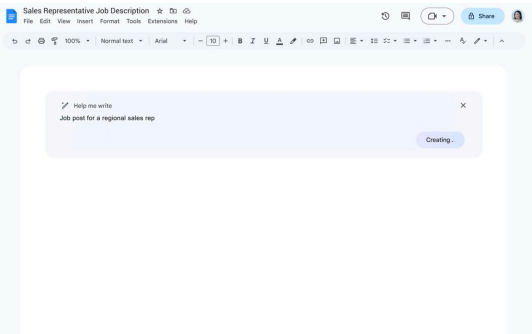
- ✓ Feature has beginner and advanced modes
- ✓ Most users need simple version
- ✓ Advanced options revealed on demand

Key Insight

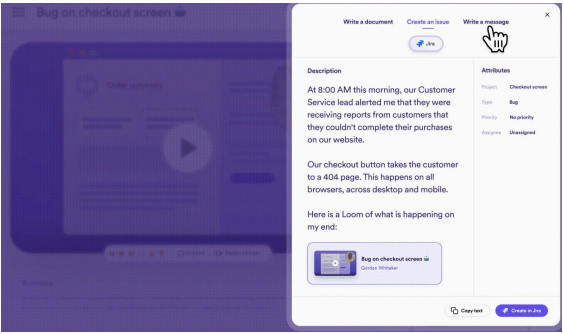
Expertise is earned. Give beginners a path forward. Power users get their tools. Same feature. Different curves.

DO ✓	DON'T ✕
Start with core task (one button, clear purpose)	Show all options at once
Show 80% features by default	Hide important features
Reveal advanced options when users are ready	Bury simple tasks in menus

Examples



Google Docs starts with basic writing tools, then progressively reveals AI features like Smart Compose



Basic video editing options are shown; advanced AI features (e.g., auto-transcription, highlights) are revealed as needed.

Next Steps

You now have 6 patterns from the best AI products in the world.

But here's the key: These aren't features to bolt onto your product. They're mindsets.

These 6 patterns work together.

Confidence + Explainability = trust.

Privacy-First + Error Recovery = safety.

Contextual Assistance + Progressive Disclosure = usability.

The teams doing AI best Claude, Apple, GitHub, Google aren't rushing to add AI everywhere. They're being thoughtful about where it actually helps. They're respecting user intelligence and autonomy. That's your competitive advantage.

Every time you design a feature or product for AI, ask:

- ✓ Is it contextually helpful? (Or just an "Ask AI" button?)
- ✓ Will users know when to trust it? (Or blindly accept?)
- ✓ What happens when it fails? (Design for recovery)
- ✓ Are we being transparent about data? (Or hiding it?)
- ✓ Can users understand why? (Or is it a black box?)
- ✓ Will this overwhelm beginners? (Or progressively empower?)

Next Steps:

1. Pick one pattern that matches your most pressing problem
2. Design for that pattern in your next feature
3. Test with real users to validate your approach
4. Learn from feedback and iterate
5. Add another pattern when you're ready

Remember: The goal isn't AI for AI's sake. The goal is creating products where AI feels like a helpful colleague, not a manipulative algorithm.

You have the patterns. You have the knowledge. Now go build something users will actually love.

Want more? We are growing a library of patterns. aiuxdesign.guide